INTEGRATION DOCUMENTATION COLLECTION SERVICE (REDIRECT)

V1.3 – 2025

I. OVERVIEW

I.1. DEFINITIONS

Terms	Description
Pay2Pay	Pay2Pay Payment Gateway
Merchant	Partners using the services of Pay2Pay
Bank	The bank where the partner's customers want to receive money
HTTPS	Secure Hypertext Transfer Protocol
TLS	Transfer Layer Security
Signature	To ensure data transmission between Merchant and Pay2Pay is secure and accurate, all data transmitted between the two parties will be encrypted and digitally signed to ensure data constraints.

I.2. DOCUMENT PURPOSE

The document describes the detailed steps necessary for partners to integrate into the Pay2Pay Payment Gateway system.

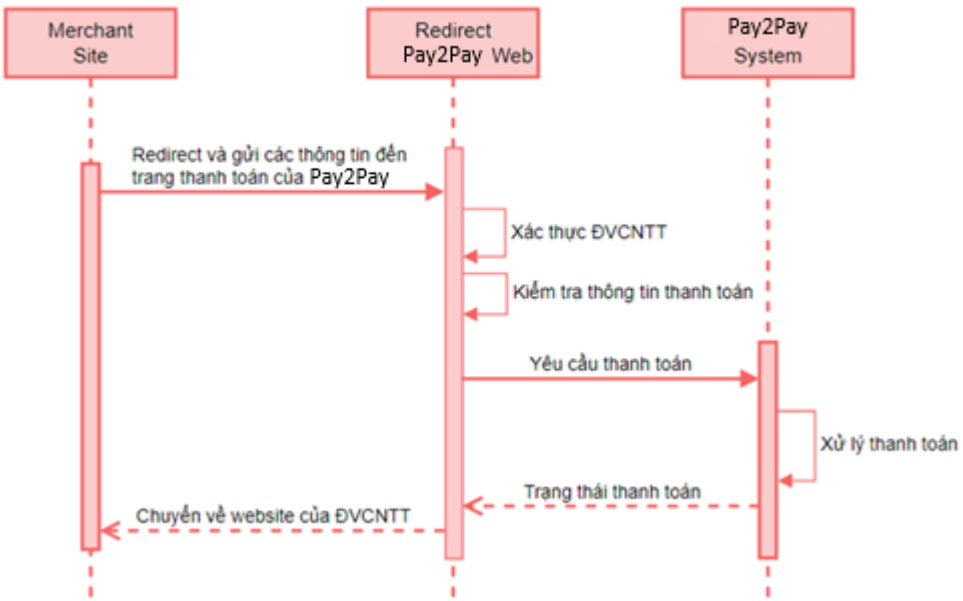
I.3. OBJECT

This document is used for partner developers to integrate into the Pay2Pay Payment Gateway system.

II. CONNECTION METHOD BY WEB REDIRECT

II.1. COMMUNICATION MODEL

The following figure describes an overview of the communication model between the Merchant's system and the Pay2Pay system.



- Pay2Pay uses HTTPS/TLS1.2 connection standard
- To ensure data transmission and reception between the two parties is secure and authentic, all data will be encrypted and digitally signed.

II.2. SIGNATURE

To ensure data transmission between Merchant and Pay2Pay is secure and authentic, all data transmission between the two parties will be encrypted and digitally signed.

ALGORITHM:

The signature is created based on the SHA256 hash algorithm

HOW TO CREATE A SIGNATURE:

The steps to create a signature are as follows:

1. Sort the data parameters in URL Query string order by alphabet.
2. Get the value of each sorted data field name and join them together.
3. Connect the above string with **merchant_key** (merchant_key is the merchant key code)
4. Apply the SHA-256 hash algorithm to the concatenated string.

Convert the hash result to Base64 format.

EXAMPLE:

The payment call link to Pay2Pay will be calculated as follows:

https://{DOMAIN}/?merchant_id=PP0000024&order_code=66595a794d32564674e2ce26&total_amount=108000&currency=VND&url_redirect=https://merchant-web.com&language=vi&timestamp=1717131896&content=Thanh+toan+don+hang+eCom

The fields have values as follows:

1. merchant_id**: PP0000024**
2. order_code**: 66595a794d32564674e2ce26**
3. total_amount**: 108000**
4. currency**: VND**
5. language**: vi**
6. timestamp**: 1717131896**
7. content**: Thanh+toan+don+hang+eCom**
8. merchant_key là key **m3arCAFqqRQuSh1tQwsJJg==**
9. url_redirect: Redirect the user to the merchant's website after the customer completes the payment

Based on the given formula, we will perform the following steps to create a signature:

- Arrange data fields order by alphabet:
  - content: **Thanh+toan+don+hang+eCom**
  - currency: **VND**
  - language: **vi**
  - merchant: **PP0000024**
  - order_code: **66595a794d32564674e2ce26**
  - timestamp: **1717131896**
  - total_amount: **108000**
  - url_redirect: https://merchant-web.com
- Append the values of each sorted data field together:
  - Thanh+toan+don+hang+eComVNDviPP000002466595a794d32564674e2ce261717131896108000https://merchant-web.com
- Append the string above with **merchant_key**
  - Thanh+toan+don+hang+eComVNDviPP000002466595a794d32564674e2ce261717131896108000https://merchant-web.comm3arCAFqqRQuSh1tQwsJJg==
- Apply the SHA-256 hash algorithm to the concatenated string
- Convert hash results to Base64 format
  - Code javascript:

```
const crypto = require('crypto');
const inputString =
'Thanh+toan+don+hang+eComVNDviPP000002466595a794d32564674e2ce261717131896108000https://merchant-web.comm3arCAFqqRQuSh1tQwsJJg==';
const hashedValue = crypto.createHash('sha256').update(inputString, 'utf-8').digest('base64');
```

So the signature of the above example would be: **GQs2Khn+ZTdC59Qqy79HZ4Mm2QKmU0MS0WVr1wS/T+Q=**
https://{DOMAIN}/?merchant_id=PP0000024&order_code=66595a794d32564674e2ce26&total_amount=108000&currency=VND&url_redirect=https://merchant-web.com&language=vi&timestamp=1717131896&content=Thanh+toan+don+hang+eCom&signature=ifixyUVrihcA9gR4Vmp26k5LJcZP8XNIS2MYDSi4GMM%3D

III. SYSTEM INFORMATION

III.1. DOMAIN

Environment	URL
Test	https://uat-checkout.pay2pay.vn
Production	https://checkout.pay2pay.vn

III.2. MERCHANT ADMIN WEB INFORMATION

Merchant will be provided with an account to login the Merchant Administration Page built by Pay2Pay to track payment transactions.

Environment	URL
Test	https://uat-merchant.pay2pay.vn
Production	https://merchant.pay2pay.vn

Merchant Administration Page login account information will be provided when the two parties proceed with the integration.

III.3. INFORMATION MERCHANT NEEDS TO PROVIDE

Merchant needs to provide the following information to Pay2Pay

Info	Condtion	Description
IP address	Mandatory	List of IP addresses wil be used to send requests to Pay2Pay. - Required on Production environment - Optional on Test environment.
ipn_url	Mandatory	URL to receive transaction results from Pay2Pay, sent immediately after confirmation from the bank.
redirect_url	Optional	URL for Pay2Pay to transfer customers to the merchant site, after the customer completes the transfer on the Pay2Pay site.
Public Key	Mandatory	The public key the merchant provides to Pay2Pay allows Pay2Pay to verify the merchant's signature when sending a request to Pay2Pay.

IV. CONNECTION INSTRUCTIONS BY REDIRECT

IV.1. CONNECTION DETAILED DESCRIPTION OF PAYMENT TRANSACTION

URL: https://{DOMAIN}
Method: GET
Query format

#	Parameter	Data Type	Condition (M - Mandatory/Option)	Description
1	merchant_id	String	M	Merchant identify
2	order_code	String	M	Purchase order
3	total_amount	Number	M	Total amount of purchase order
4	currency	String	M	Currency code (Ex: vnd, usd,…)
5	language	String	M	Language (Ex: vi, en,...)
6	timestamp	Long	M	Time to request payment information (Unix time from 00:00:00 January 1, 1970)
7	content	String	M	Payment content of the purchase order
8	signature	String	M	Signature according to SHA256 algorithm. Refer to II.2

Example: Link to call payment from the merchant/seller website after integrating payment with Pay2Pay's payment page
URL:
https://{DOMAIN}/?merchant_id=PP0000001&order_code=66430755e4a8aac0f0f89808&total_amount=699000&currency=VND&language=vi&timestamp=1715668821&content=Thanh+toan+don+hang+eCom&signature=AO42dpZMMjdZuWAOyqSBWYDlrhvIyHCb8HfR%2FvdNXHM%3D

IV.2. IPN – Instant Payment Notification

Merchant will receive results from Pay2Pay for the transaction flow via IPN (POST method). This result is captured in requestBody.
Method: POST
Request Header

#	Parameter	Data Type	Description
1	P-API-KEY	String	Merchant's Api key
2	P-SIGNATURE	String	Signature generate from whole request body & secret key

Request Body

#	Parameter	Data Type	Description
1	merchantId	String	Merchant ID
2	orderId	String	Purchase Order ID generated by merchant (unique)
3	txnId	String	Pay2Pay Transaction ID
4	amount	Number	Amount of purchase order
5	status	String	Status code (SUCCESS/FAIL/PROCESSING)
6	code	String	Transaction code, Please refer to V. List of API response code
7	message	String	Description of transaction status
8	txnDate	String	Transaction time
9	secretSenderBankRefNumber	String	Base64 encoded string of sender's bank account number
10	senderBankRefName	String	Bank account name of sender
11	senderBankId	String	Bank Id of sender
12	senderBankName	String	Bank Name of sender
13	senderBinCode	String	Bank Bin Code of sender

Steps to Create Signature:

Step 1: convert request body to JSON string

{"merchantId":"PP0000001","orderId":"66433ebbe4a8aac0f0f8a5f6","txnId":"202405140004184","amount":92000,"code":"SUCCESS","message":"Transaction is successful.","txnDate":"20240514173722"}

Step 2: append secretKey value to the result of **Step 1**

{"merchantId":"PP0000001","orderId":"66433ebbe4a8aac0f0f8a5f6","txnId":"202405140004184","amount":"92000","code":"SUCCESS","message":"Transactionissuccessful.","txnDate":"20240514173722","secretSenderBankRefNumber":"MjAzNDAzMDQ0MDAwMA==","senderBankRefName":"NGUYENVANA","senderBankId":"VCB"}**<secretKey value>**

Step 3: Apply the SHA-256 hash algorithm to string value of Step 2

Step 2's Value:
{"merchantId":"PP0000001","orderId":"66433ebbe4a8aac0f0f8a5f6","txnId":"202405140004184","amount":"92000","code":"SUCCESS","message":"Transactionissuccessful.","txnDate":"20240514173722","secretSenderBankRefNumber":"MjAzNDAzMDQ0MDAwMA==","senderBankRefName":"NGUYENVANA","senderBankId":"VCB"}MyScretKey

Signature value: use this value to verify with header signature
7Vi93pdDN3B6AQte/IEQ7ILzqKidXuQtHiN/HozZ/Mc=

Refer link to check & get snippet code: https://static.pay2pay.vn/tools/ipn_validate

Example:

Data Sample

Request Header:

P-API-KEY : 123456
P-SIGNATURE: vlQBS3i/3iD/VF6x3GgHMGgEzF2Vke+JDz9h9B3KcCc=

Request Body:

{
"merchantId": "PP0000001",
"orderId": "66433ebbe4a8aac0f0f8a5f6",
"txnId": "202405140004184",
"amount": "92000",
"status": "SUCCESS",
"code": "SUCCESS",
"message": "Transaction is successful.",
"txnDate": "20240514173722",
"secretSenderBankRefNumber": "MjAzNDAzMDQ0MDAwMA==",
"senderBankRefName": "NGUYEN VAN A",
"senderBankId": "VCB",
"senderBankName":"VIETCOMBANK - NH Ngoai Thuong Viet Nam",
"senderBinCode":" 970436"
}

IV.3. URL Params Callback

Example:

https://{YOUR_DOMAIN}?status=SUCCESS&merchantId=PP0000249&orderId=9d6f70cf3eb531da6f9f3f09&txnId=202511290403784&amount=100000&code=SUCCESS&message=Giao+d%E1%BB%8Bch+th%C3%A0nh+c%C3%B4ng&txnDate=20251129&signature=glhXJNxSModXayh6E39f%2FtXBXccNkUZl3V4P6231m%2FQ%3D

Step 1: parsing params:

'status':	SUCCESS
'merchantId':	PP0000249
'orderId':	9d6f70cf3eb531da6f9f3f09
'txnId':	202511290403784
'amount':	100000
'code':	SUCCESS
'message':	Giao dịch thành công
'txnDate':	20251129

Step 2: Combine params value into a string

SUCCESSPP00002499d6f70cf3eb531da6f9f3f09202511290403784100000SUCCESSGiao dịch thành công20251129

Step 3: Using sha256 to create signature

Sha256("SUCCESSPP00002499d6f70cf3eb531da6f9f3f09202511290403784100000SUCCESSGiao dịch thành công20251129") => glhXJNxSModXayh6E39f/tXBXccNkUZl3V4P6231m/Q=

Step 4: Compare with signature param value to validate

#	Parameters	Data Type	Description
1	status	String	Result transaction
2	merchantId	String	Merchant ID
3	orderId	String	Purchase Order ID generated by merchant (unique)
4	txnId	String	Pay2Pay Transaction ID
5	amount	Number	Amount of purchase order
6	code	String	Result code
7	message	String	Description of transaction status
8	txnDate	String	Transaction time
9	signature	String	Signature according to SHA256 algorithm. Refer to II.2

V. LIST OF API STATUS CODES

Status Codes indicate whether a transaction has succeeded, failed, or is still being processed. Merchants use these codes to decide how to handle each transaction, such as confirming success, waiting for completion, or retrying when necessary.

No	Status	Description
1	SUCCESS	The transaction has been completed successfully. The final result is confirmed, and no further action is required. Applicable to both collection and payout.
2	PROCESSING	The transaction has been received and is currently being processed by the bank or payment provider. The final status will be updated later through callback.
3	FAIL	The transaction could not be completed. This may be due to incorrect information, insufficient balance, exceeded limits, timeout, or external bank rejection. Final state—no further processing will occur.
4	INIT	The transaction has been initialized and recorded in the system, but no processing has started yet. This status indicates the initial creation stage before being sent for processing.

5	CANCELED	The transaction has been canceled intentionally by the user or the system before completion. No further processing will occur, and no funds will be deducted or transferred.
6	SUSPECT	The transaction has been flagged as suspicious due to abnormal behavior, potential fraud, or risk control rules. Further verification or manual review is required before determining the final result.

VI.

VII. LIST OF API RESPONSE CODES

Response Codes provide specific error details for each API request. They help merchants identify why a request failed and offer accurate messages or guidance so users can correct the issue before retrying.

No	Code	Message
1	SUCCESS	Success code
2	ERR_BANK_SYSTEM	Error system
3	ERR_BANK_TIMEOUT	Error system
4	ERR_BANK_ACCOUNT_IS_NOT_CORRECT	Account name or number is not correct
5	ERR_BANK_REQUEST_ID_ALREADY_EXIST	The request id already exists
6	ERR_BANK_INVALID_PARAMETER	Invalid parameter
7	ERR_BANK_BANK_INFORMATION_NOT_FOUND	Bank information not found
8	ERR_BANK_TRANSACTION_FAIL	Transaction failed
9	ERR_BANK_NOT_RECEIVE_RESPONSE_FROM_BANK	Did not receive response from bank to transfer money
10	ERR_BANK_CURRENT_BANK_DISCONTINUITY	The current beneficiary bank is discontinuity
11	ERR_BANK_ISSUING_BANK_REFUSES_PROCESS_REQUEST	Issuing bank refuses to process this request
12	ERR_BANK_PAYING_BANK_REFUSES_PROCESS_REQUEST	The paying bank refuses to process this request
13	ERR_BANK_MERCHANT_ACTIVE_BANK_VA_INFO_DOES_NOT_EXIST	Merchant's account haven't declared
14	ERR_BANK_TRANSACTION_DOES_NOT_EXIST	Transaction does not exist
15	ERR_BANK_TRANSACTION_STATUS_INVALID	The status of the transaction is invalid
16	ERR_SYSTEM	system error.
17	ERR_TRANSACTION_MERCHANT_INVALID	invalid merchant.
18	ERR_DATA_INVALID	invalid data.
19	ERR_TOTALAMOUNT_INVALID	Invalid Total amount.
20	ERR_TRANSACTION_METHOD_INVALID	Invalid payment method.
21	ERR_TRANSACTION_ISSUER_INVALID	Invalid issuer.
22	ERR_TRANSACTION_CURRENCY_INVALID	Invalid currency amount.
23	ERR_TRANSACTION_INIT_DATA_NOT_FOUND	Init data not found.
24	ERR_TRANSACTION_NOT_FOUND	Transaction not found.
25	MERCHANT_INVALID	invalid merchant.
26	ERR_MERCHANT_DOES_NOT_EXIST	Merchant does not exist
27	ERR_MERCHANT_FEE_INVALID	Invalid transaction fee
28	ERR_MERCHANT_TRANSACTION_INVALID	Invalid transaction
29	ERR_MERCHANT_CORE_SUSPECT	System error. Please try again in a few minutes
30	ERR_MERCHANT_CORE_TIMEOUT	System error. Please try again in a few minutes
31	ERR_MERCHANT_BANK_TIMEOUT	System error. Please try again in a few minutes

32	ERR_MERCHANT_BANK_SUSPECT	System error. Please try again in a few minutes

VIII. CODE SAMPLE

Sample code creates a signature based on the SHA256 hash algorithm

1. **PHP**

```
function computeSha256HashBase64($inputString) {
  // Compute the SHA-256 hash of the input string
  $hash = hash('sha256', $inputString, true);
  // Encode the resulting hash in Base64
  $base64Hash = base64_encode($hash);
  return $base64Hash;
}
```

2. **C#**

```
public static string ComputeSha256HashBase64(string inputString)
  {
    byte[] inputBytes = Encoding.UTF8.GetBytes(inputString);
    // Create a SHA256 instance
    using (SHA256 sha256 = SHA256.Create())
    {
      byte[] hashBytes = sha256.ComputeHash(inputBytes);
      string base64Hash = Convert.ToBase64String(hashBytes);
      return base64Hash;
    }
  }
```

3. **ReactJs**

```
import CryptoJS from 'crypto-js';
function computeSha256HashBase64(inputString) {
  // Compute SHA-256 hash
  const hash = CryptoJS.SHA256(inputString);

  // Convert the hash to a Base64 string
  const base64Hash = CryptoJS.enc.Base64.stringify(hash);

  return base64Hash;
}
```

4. **Java**

```
public static String computeSha256HashBase64(String inputString) {
    try {
      MessageDigest digest = MessageDigest.getInstance("SHA-256");
      byte[] hashBytes = digest.digest(inputString.getBytes(StandardCharsets.UTF_8));
      String base64Hash = Base64.getEncoder().encodeToString(hashBytes);
      return base64Hash;
    } catch (NoSuchAlgorithmException e) {
      throw new RuntimeException(e);
    }
  }
```